



IIC2343 - Arquitectura de Computadores (I/2026)

Guía de ejercicios: Coherencia de Caché

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl), Ignacio Gajardo (gajardo.ignacio@uc.cl)

Pregunta 1: Preguntas Conceptuales

- a) Explique la diferencia entre los esquemas UMA y NUMA en cuanto a cómo los procesadores acceden a la memoria.

Solución: En UMA todos los procesadores acceden de la misma forma a la memoria, a través de un único bus o un crossbar switch. En NUMA los procesadores se reparten en nodos, cada uno con acceso rápido a su memoria local y acceso a la memoria de otros nodos (remota) con mayor latencia.

- b) ¿Por qué una política *write-through* en general no genera problemas de coherencia, y por qué aun así se prefiere *write-back* a pesar de introducirlos?

Solución: Con *write-through* cada escritura se propaga a memoria principal, por lo que esta queda consistente para todos los procesadores. El problema es que cada escritura implica un overhead de comunicación: hay que transmitir el dato por el bus compartido, lo que lo satura y aumenta el tiempo de ejecución.

- c) ¿Qué hace un controlador que implementa *snooping* al detectar una escritura a una dirección que contiene, y por qué este mecanismo se utiliza menos?

Solución: Al detectar una solicitud de escritura a una dirección que posee, actualiza de inmediato el contenido de su caché con los datos de la solicitud. Se usa menos porque ese monitoreo y actualización constante obliga a leer cada escritura del bus, lo que aumenta el tráfico en el bus compartido de datos y, con ello, la latencia de los accesos a memoria.

- d) ¿En qué consiste el *bus snooping* y qué rol juega el controlador de caché en él?

Solución: Consiste en que cada controlador de caché revisa, a través del bus compartido, las solicitudes de lectura y escritura que realizan sus contrapartes. Gracias a esa observación, cada controlador sabe qué están haciendo las demás cachés sobre las líneas que él posee y puede gatillar los cambios de estado que define el protocolo; por eso es el mecanismo mediante el cual se adoptan los protocolos de coherencia.

- e) ¿Cuál es el problema del protocolo MSI que motiva el desarrollo de MESI, y cómo lo resuelve el nuevo estado *Exclusive*?

Solución: El problema de MSI es que usa el estado *Shared* aunque solo una caché posea el bloque, por lo que una escritura sobre esa línea igual gatilla un BusRdX innecesario, ocupando el bus. MESI lo resuelve con *Exclusive*, pudiendo distinguir el caso en que el contenido está en una sola caché y sin modificaciones, por lo que no es necesario emitir la señal BusRdX y así usar menos el bus de datos.

- f) Si una línea está en estado *Exclusive* y el procesador realiza una escritura, ¿a qué estado pasa y por qué no necesita emitir una señal al bus compartido?

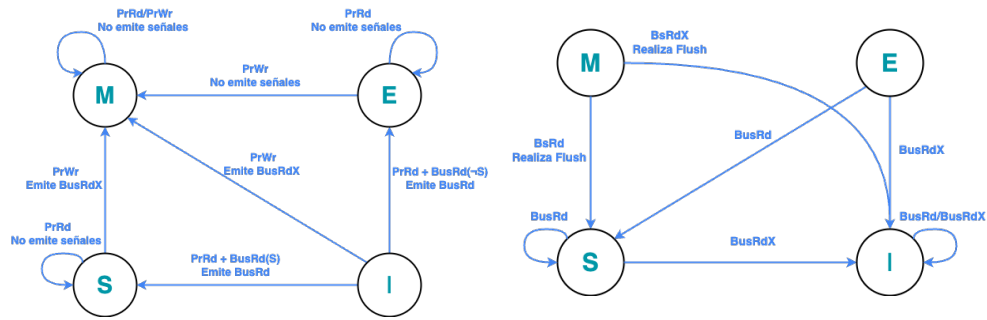
Solución: Pasa a estado *Modified*. No emite señal al bus porque, al estar en *Exclusive*, se sabe que ninguna otra caché posee el bloque, por lo que no hay copias que invalidar.

Pregunta 2: Protocolo MESIF (P4 I3 2024-1)

El protocolo MESIF consiste en una extensión de MESI que incluye un nuevo estado **F** (*Forward*). Este protocolo presenta las siguientes diferencias respecto a MESI:

- Si dos o más caché poseen una misma copia de un bloque sin cambios de la memoria compartida, **solo una** de ellas se encarga de transmitir la línea a otras CPU que soliciten el contenido. Esta caché será la que poseerá la línea en estado **F**.
- La caché **con la copia más reciente** se encarga de realizar la transmisión. Una vez que la caché transmite un bloque, su línea pasa de estado **F** a **S**. La que recibe la copia, en cambio, pasa a estado **F** y se vuelve la nueva encargada de realizar las transmisiones.
- Si la línea de una primera caché se encuentra en estado **E** y una segunda caché otra solicita una copia del bloque, entonces la línea de la primera pasa de estado **E** a **S**, mientras que la línea de la segunda pasa a estado **F**. En este caso, el bloque no es transmitido entre las dos caché, sino que se obtiene directamente de la memoria principal.

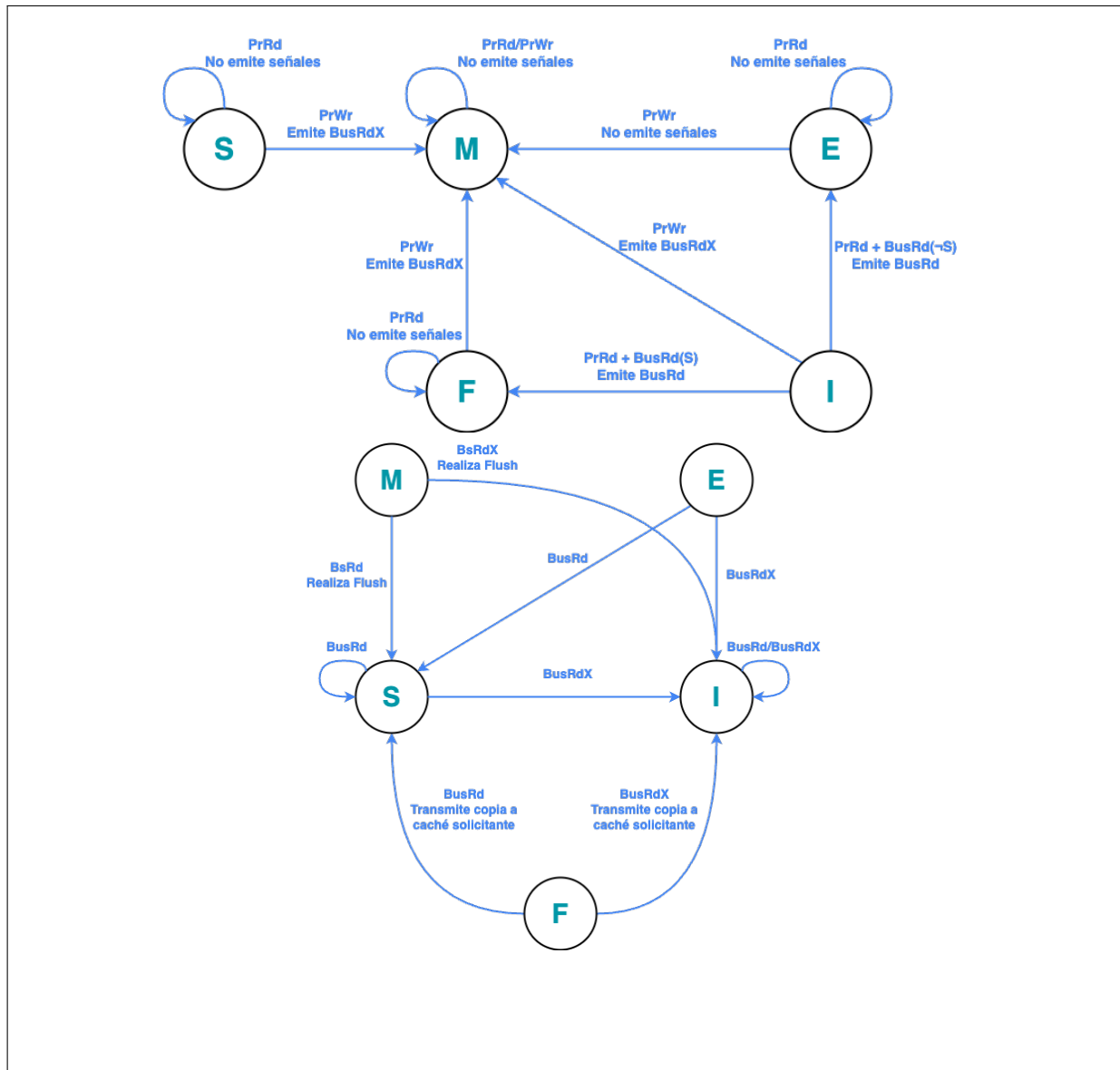
Modifique los diagramas de estados adjuntos para incluir el nuevo estado **F** y los cambios correspondientes según los eventos del procesador (**PrRd**, **PrWr**) y del bus compartido (**BusRd**, **BusRdX**). No es necesario que indique las señales que se emiten en cada cambio de estado, pero sí debe señalar el evento que gatilla el cambio de un estado a otro.



Solución: Se indicarán los cambios de estado de **F** según cada tipo de evento:

- **PrRd:** El procesador actual solicita una lectura sobre la línea en estado **F**. Esto no implica transmisión alguna de la copia a otra caché, por lo que se mantiene en este estado. No es necesario que emita señales.
- **PrWr:** El procesador actual solicita una escritura sobre la línea en estado **F**. Esto implica cambiar a estado **M** y, al mismo tiempo, invalidar las copias de otras caché, por lo que se emite la señal **BusRdX** con ese fin.
- **BusRd:** Un procesador distinto del actual solicita una lectura sobre la línea en estado **F**. La caché del procesador actual transmite su copia. Al hacerlo, cambia a estado **S** al no ser la que posee la copia más reciente. No emite señales ni realiza *flush*.
- **BusRdX:** Un procesador distinto del actual solicita una escritura sobre la línea en estado **F**. La caché del procesador actual se encarga de transmitir su copia. Al hacerlo, cambia a estado **I** al poseer ahora una copia que no posee las últimas modificaciones. No emite señales ni realiza *flush*.

El resto de los estados se mantiene sin cambios **salvo por I**: Si un procesador solicita la lectura (**PrRd**) de un bloque a ser almacenado en una línea en estado **I**, y esta copia existe en más memorias caché (**BusRd(S)**), entonces ya no pasará a estado **S**, sino que pasará a estado **F** al poseer la copia más reciente. Debe seguir emitiendo la señal **BusRd** ya que, a través de ella, la caché que se encuentre en estado **F** podrá transmitir su copia y pasar a estado **S**. A continuación, se muestran los diagramas resultantes:



Pregunta 3: Coherencia de Caché (P4 I3 2024-2)

Suponga que posee una arquitectura MIMD de memoria compartida con 3 CPU que poseen su propia caché y que siguen el protocolo MESI para asegurar consistencia. La memoria contiene los siguientes datos de variables; mientras que CPU0, CPU1 y CPU2 ejecutan los siguientes programas:

Dirección	Label	Valor
0x00	var1	255
0x01	var2	253
...	...	...
0xFE		0
0xFF		0

// CPU0	// CPU1	// CPU2
MOV A, (var1)	MOV A, 0	MOV B, 0
MOV B, (var2)	NOP	PUSH B
AND B,A	PUSH A	INC B
INC B	POP B	PUSH B
MOV B, (B)		POP A
MOV (B), B	MOV B,(B)	

Asumiendo que cada dirección se almacena en una línea distinta, y que el contador SP posee un valor inicial igual a 255 en **todas** las CPU, indique el estado de cada línea de las caché (M/E/S/I) para cada ciclo completando la tabla adjunta, asumiendo que todas las líneas parten en estado I (ciclo 0):

	CPU0				CPU1				CPU2			
Ciclo	0x00	0x01	0xFE	0xFF	0x00	0x01	0xFE	0xFF	0x00	0x01	0xFE	0xFF
0	I	I	I	I	I	I	I	I	I	I	I	I
1												
2												
3												
4												
5												
6												

Solución: A continuación se muestra la tabla final esperada por cada iteración:

	CPU0				CPU1				CPU2			
Ciclo	0x00	0x01	0xFE	0xFF	0x00	0x01	0xFE	0xFF	0x00	0x01	0xFE	0xFF
0	I	I	I	I	I	I	I	I	I	I	I	I
1	E	I	I	I	I	I	I	I	I	I	I	I
2	E	E	I	I	I	I	I	I	I	I	I	M
3	E	E	I	I	I	I	I	M	I	I	I	I
4	E	E	I	I	I	I	I	M	I	I	M	I
5	E	E	S	I	I	I	I	M	I	I	S	I
6	S	M	S	I	S	I	I	M	I	I	S	I

Para llegar al estado final correcto, es importante haber tomado en cuenta lo siguiente:

- Si se realiza la operación AND B,A con 8 bits para cada registro, se llega al resultado B = 253, por lo que la CPU0 luego accede a la dirección de memoria 254 = 0xFE considerando la ejecución de la instrucción INC B.
- La ejecución de POP A y POP B toma dos ciclos. La lectura del valor de memoria se realiza en el **segundo ciclo**, dado que el primero lo único que hace es incrementar el valor de SP.
- Al ser SP parte de una CPU, su valor es **independiente** de lo que ejecuten otras. Es decir, que una CPU realice un PUSH o un POP no afecta el valor de SP de otra.

Pregunta 4: Coherencia de Caché (P4 Sección 3 I3 2024-1)

Considere la siguiente memoria. Esta es compartida por 3 CPUs con paralelismo UMA. Para mantener la coherencia entre sus caché, se usa el protocolo MESI.

Memoria compartida

Dir	Label	Valor
0	x	1
1	y	2
2	arr	3
3		4
4		5

CPU 0

```
MOV B, arr
INC B
MOV A, (B)
MOV (x), A
MOV A, 3
MOV B, (4)
ADD A, B
MOV (4), A
```

CPU 1

```
MOV A, (x)
MOV B, (y)
ADD A, B
MOV (3), A
MOV A, 2
MOV B, (x)
MOV (3), B
ADD A, B
```

CPU 2

```
MOV B, arr
MOV A, 0
ADD A, (B)
INC B
ADD A, (B)
INC B
MOV (y), A
MOV B, (3)
```

Puede asumir que las líneas de la caché son de una palabra y todas las líneas de las caché privadas parten en estado inválido. Complete las siguientes tablas a partir de la ejecución de los programas en las tres CPU. En cada entrada, indique si el valor fue **modificado** (M), es **exclusivo** (E), **compartido** (S) o **inválido** (I).

Caché - CPU 0					
Inst	x	y	arr[0]	arr[1]	arr[2]
1					
2					
3					
4					
5					
6					
7					
8					

Caché - CPU 1					
Inst	x	y	arr[0]	arr[1]	arr[2]
1					
2					
3					
4					
5					
6					
7					
8					

Caché - CPU 2					
Inst	x	y	arr[0]	arr[1]	arr[2]
1					
2					
3					
4					
5					
6					
7					
8					

Solución:

A continuación, se muestra el estado de la caché de cada CPU posterior a la ejecución de cada una de las instrucciones de los programas:

Caché - CPU 0					
Inst	x	y	arr[0]	arr[1]	arr[2]
1	I	I	I	I	I
2	I	I	I	I	I
3	I	I	I	E	I
4	M	I	I	I	I
5	M	I	I	I	I
6	S	I	I	I	E
7	S	I	I	I	E
8	S	I	I	I	M

Caché - CPU 1					
Inst	x	y	arr[0]	arr[1]	arr[2]
1	E	I	I	I	I
2	E	E	I	I	I
3	E	E	I	I	I
4	I	E	I	M	I
5	I	E	I	S	I
6	S	E	I	S	I
7	S	I	I	M	I
8	S	I	I	S	I

Caché - CPU 2					
Inst	x	y	arr[0]	arr[1]	arr[2]
1	I	I	I	I	I
2	I	I	I	I	I
3	I	I	E	I	I
4	I	I	E	I	I
5	I	I	E	S	I
6	I	I	E	S	I
7	I	M	E	I	I
8	I	M	E	S	I

Pregunta 5: Coherencia de Caché (P4 Sección 4 I3 2024-1)

Suponga que posee una arquitectura MIMD de memoria compartida con 3 CPU que poseen su propia caché y que siguen el protocolo MESI para asegurar consistencia. La memoria contiene los siguientes datos de variables; mientras que CPU0, CPU1 y CPU2 ejecutan los siguientes programas:

Dirección	Label	Valor
0x00	arr	2
0x01		0
0x02		1
...	...	...
0xFE		0
0xFF		0

// CPU0	// CPU1	// CPU2
MOV B,arr	MOV A,2	MOV A,5
INC B	PUSH A	MOV B,1
MOV B,(B)	POP A	PUSH A
MOV (B),B		PUSH B

Asumiendo que cada dirección se almacena en una línea distinta, y que el contador SP posee un valor inicial igual a 255 en **todas** las CPU, indique el estado de cada línea de las caché (M/E/S/I) para cada ciclo completando la tabla adjunta, asumiendo que todas las líneas parten en estado I (ciclo 0):

	CPU0					CPU1					CPU2				
Ciclo	0x00	0x01	0x02	0xFE	0xFF	0x00	0x01	0x02	0xFE	0xFF	0x00	0x01	0x02	0xFE	0xFF
0	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
1															
2															
3															
4															

Solución: A continuación se muestra la tabla final esperada por cada iteración:

	CPU0					CPU1					CPU2				
Ciclo	0x00	0x01	0x02	0xFE	0xFF	0x00	0x01	0x02	0xFE	0xFF	0x00	0x01	0x02	0xFE	0xFF
0	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
1	I	I	I	I	I	I	I	I	I	I	I	I	I	I	I
2	I	I	I	I	I	I	I	I	I	M	I	I	I	I	I
3	I	E	I	I	I	I	I	I	I	I	I	I	I	I	M
4	M	E	I	I	I	I	I	I	I	S	I	I	I	M	S

Para llegar al estado final correcto, es importante haber tomado en cuenta lo siguiente:

- **MOV B,arr no es una instrucción de lectura de memoria.** Solo guarda la dirección de arr en B.
- La ejecución de **POP A** toma dos ciclos. La lectura del valor de memoria se realiza en el **segundo ciclo**, dado que el primero lo único que hace es incrementar el valor de SP.
- Al ser SP parte de una CPU, su valor es **independiente** de lo que ejecuten otras. Es decir, que una CPU realice un **PUSH** o un **POP** no afecta el valor de SP de otra.